

Natural Language Processing

Assignment 1

Total Mark: 100%

Building a Parser for DNA/RNA Sequences

Estimated completion time: ~2 hours

Introduction

In bioinformatics, FASTA files contain biological sequence data and metadata in a simple format. Each sequence record starts with a header line (prefixed by >), followed by one or more lines of sequence data. A typical FASTA record looks like this:

```
>Sequence_ID or description
ACGTACGTACGT
AACGTNACGTTT
>Another_Sequence_ID or description
AUGUACGUACGUACGUA
```

- Lines starting with > are **headers**.
- The lines immediately following a header belong to that sequence (until the next header or the end of the file).
- **DNA** sequences typically contain A, C, G, T, **RNA** sequences contain A, C, G, U, and many bioinformatics pipelines also allow for **ambiguous nucleotides** (like N, R, Y, etc.) which represent uncertain bases.

What You'll Do

Part 1: Parsing a FASTA-like file

Part 2: Identifying start/stop codons and open reading frames (ORFs)

Part 3: Calculating nucleotide frequencies and generating summary statistics

By the end of this assignment, you'll have reinforced your understanding of Python string manipulation, regular expressions, and the basic structure of genomic data.

Question 1: Parsing FASTA with Regex (30%)

Part 1: Parsing FASTA with Regex

1. Input File
You will be given a file (or text) named `test_part1_in.fasta` with multiple records. Each record has:
 - A header line beginning with `>` (e.g., `>Seq_1`).
 - One or more lines of sequence data (until the next `>` or the end of file).
2. Merge Multiline Sequences
 - Read the file line by line.
 - Use a regex (or string check) to detect header lines beginning with `>`.
 - Accumulate the following lines (until you hit another `>` or EOF) into a single contiguous string (remove newlines/spaces).
3. Classify Sequences as DNA, RNA, or Invalid
 - DNA: Contains T (and possibly ambiguous letters like N, R, etc.) but no U.
 - RNA: Contains U (and possibly ambiguous letters) but no T.
 - Invalid: If the sequence has both T and U, or if it lacks any valid nucleotides altogether.

Deliverable for Part 1

- A script that **reads** the fasta file, **parses** headers and sequences, and **prints**:
 - The header
 - The merged sequence (on one line)
 - A label: **DNA**, **RNA**, or **Invalid** (based on your classification rules).

Example (pseudo) output for each record:

```
>Sequence_1
ACGTNACGTN
Type: DNA
```

```
>Sequence_2
AUGNUGC
Type: RNA
```

Question 2: Locating Start/Stop Codons and Open Reading Frames (30%)

In genetics, an **open reading frame (ORF)** is a stretch of nucleotides that starts with a **start codon** and ends with a **stop codon**. For simplicity, we'll focus on the standard codons:

- **DNA** Start Codon: ATG
- **DNA** Stop Codons: TAA, TAG, TGA
- **RNA** Start Codon: AUG
- **RNA** Stop Codons: UAA, UAG, UGA

Task: For each valid DNA or RNA sequence in the given test file *test_part2_in.fasta*, locate **all possible** start→stop segments that appear in-frame. (We'll assume reading frame 1 for simplicity: codons start at indices 0, 3, 6, etc.).

You can either:

- (Regex Approach) Craft a regex that matches the pattern (ATG (codons in steps of 3) (TAA|TAG|TGA)) for DNA (similarly for RNA).
- (Iterative Approach) For each valid sequence:
 1. Locate all occurrences of the start codon.
 2. From each start index, move in steps of 3 looking for a stop codon.
 3. If you encounter ambiguous letters in a critical position of a codon (e.g., A?G), treat that codon as non-matching.

Report:

- The **number** of ORFs found in each sequence.
- The **positions** of each ORF start and stop (e.g., index in the string).
- Optionally, print the **subsequence** representing each ORF.

2. Handling ambiguous letters:

- If an ambiguous letter occurs in a codon you're analyzing, you should treat it as non-matching.

Clarification Note: if **ATN** is encountered (where **N** is ambiguous), treat the entire codon as invalid for a start or stop codon.

Deliverable for Part 2

- Extend your script to **locate** all possible ORFs. For each sequence, **print**:
 - The total number of ORFs found.
 - The list of (start_index, stop_index) pairs or (start_codon, stop_codon, subsequence).

Example:

```
>Sequence_1
ACGTNATGACGTAA
Type: DNA
Found 1 ORF:
- Start at 4, Stop at 12, ORF: ATGACGTAA
```

Question 3: Nucleotide Frequencies & Summary Report (40%)

Finally, we want some **stats** about our valid sequences:

1. **Nucleotide Frequencies:**
 - For each valid sequence (DNA or RNA), count how many times each **core** nucleotide (A, C, G, T/U) appears. Ignore ambiguous nucleotides in the count or show them separately.
2. **Sequence Length Distribution:**
 - Collect the lengths of **all valid DNA** sequences, compute the **mean** length.
 - Do the same for **all valid RNA** sequences.
3. **Output:**
 - A **per-sequence** breakdown: Header, Type, Length, Freq of {A,C,G,T/U,(ambiguous)}, number of ORFs found, etc.
 - A **final summary** at the end of the run:
 - **Total** number of valid DNA sequences.
 - **Total** number of valid RNA sequences.
 - **Mean length** of DNA sequences.
 - **Mean length** of RNA sequences.
 - **Number of invalid sequences.**

Deliverable for Part 3

- Extend your script to produce a structured printout for each sequence with:
 - Header
 - Type (DNA/RNA)
 - Length
 - Nucleotide counts
 - Number of ORFs found

- Finally, produce the summary statistics.

Example excerpt:

```
>Sequence_2
Type: RNA
Length: 15
Counts: A=4, C=3, G=2, U=5, ambiguous=1
Number of ORFs: 2
```

```
---SUMMARY---
Valid DNA sequences: 3
Mean DNA length: 48
Valid RNA sequences: 2
Mean RNA length: 30
Invalid sequences: 1
```

Submission Guidelines

1. **Python Script:** Submit a single Python script.
2. **README:** Briefly describe:
 - How to run your code (python advanced_dna_rna_parser.py)
 - Any assumptions you made for ambiguous letters or ORF logic.
 - A short note on how you used regex for validation/parsing.
3. **Sample Output:** Include the console output when you run your script on the provided test files.

Important Notes: (Marking will take these into account)

- Include comments in the code.
- Use meaningful variable names and function.
- *Recommended:* create unit test (sample tests) for your functions.

PLEASE zip all files and submit a single zip file including your code, report, result and data files.

Name your zip file as: firstname.lastname.HW<number>.zip

Example: John.Doe.HW1.zip